מבוא למדעי המחשב 67101 – סמסטר ב' 2024

תרגיל 3 – לולאות

להגשה בתאריך **30/05/2024** בשעה 22:00

מבוא

בתרגיל זה נתרגל שימוש בלולאות ומשתנים. אנא קראו את הסעיפים הבאים **לפני תחילת העבודה**:

- בתרגיל זה נעבוד בקובץ יחיד בשם ex3.py.
- חתימות הפונקציות (שם הפונקציה והפרמטרים שלה) ב-ex3.py צריכות להיות זהות **במדויק** לחתימות המתוארות במשימות (היזהרו מ-copy – paste מקובץ זה, כתבו בעצמכם).
- לפני מימוש פונקציה, קראו את כל השאלה. השאלות כוללות דוגמאות והערות למימוש.
- ניתן להוסיף פונקציות נוספות וניתן להשתמש בשאלות מאוחרות יותר בפונקציות שמומשו קודם.
- **סגנון:** הקפידו על תיעוד נאות ובחרו שמות משתנים משמעותיים. הקפידו להשתמש בקבועים (שמות משתנים באותיות גדולות), על פי ההסברים שנלמדו, ורק אם יש בכך צורך.
- **שימוש בבלים שלא נלמדו בקורס:** התרגיל ניתן למימוש קצר ויעיל מאוד, בעזרת כל מיני פונקציות מובנות וספריות מוכנות מראש (למשל numpy). לכן **(!)אסור(!)** להשתמש בהן: בתרגיל זה נרצה לתרגל שימוש בלולאות, ואם נשתמש בפונקציות שמייתרות את השאלה אז לא נשיג את מבוקשנו. באופן דומה, אין להשתמש בכלי AI (כמו ChatGPT ודומיו). למשל את הפונקציה ב"מימוש מכפלה פנימית" ניתן לממש ב-2 שורות קוד (כולל החתימה) וללא לולאות. לכן אם אין בפתרון שלכם לולאה, נורה אדומה צריכה להידלק. במקרה של ספק, יש לשאול בפורום התרגיל.
  - **אין להשתמש באף שאלה בכלי של list comprehension**.
  - **מותר ורצוי להשתמש בפונקציה append**.
  - **אסור למילה "import" להופיע באף קובץ py. בתרגיל זה (שימוש בה עלול לגרור ציון 0).**
- שימו לב מתי יש לקבל קלט מהמשתמש בעזרת הפונקציה input (השאלה הראשונה בלבד) ומתי הקלט מתקבל כארגומנט בעת קריאה לפונקציה.
- בכל שאלה מפורט מה ניתן להניח על הקלט - אין צורך לבצע בדיקות תקינות שלא במסגרת ההנחה.
- בכל הסעיפים, כאשר פונקציה צריכה להחזיר ערך, הכוונה היא לשימוש במילה השמורה **return**. בפרט **הכוונה אינה להדפיס למסך בעזרת הפונקציה print**. שימו לב, **אין להדפיס למסך כל הודעה מלבד ההודעות הנדרשות במפורש**.
- בפונקציות המקבלות רשימות כקלט – **אין לשנות** את רשימות הקלט עצמה.
- שימו לב כי אתם לא נדרשים לקרוא לפונקציות שכתבתם (מלבד במימוש פונקציות אחרות).
- בכל מטלה קראו את כל השאלה, כולל הערות למימוש, לפני כתיבת הפתרון (לפעמים ההערות נחתכות לעמוד הבא).

חלק א' - קבלת קלט מהמשתמש

1. קבלת רשימה בקלט

הקדמה:

הפונקציה **input** מקבלת שורת קלט מהמשתמש וממירה אותה למחרוזת אותה ניתן לשמור לתוך משתנה. לדוגמה, בעת הרצת קטע הקוד הבא, התכנה תמתין לקלט מהמשתמש והקלט אותו יכניס המשתמש (למשל, המחרוזת "Hello") יישמר לתוך המשתנה string_from_user. במשתנה זה ניתן להשתמש, לדוגמה על מנת להדפיס את ערכו (למסך תודפס המחרוזת "Hello").

```
string_from_user = input()

print(string_from_user)
```

הגדרות:

- "רשימה ריקה" – רשימה שאינה מכילה איברים. היא עדיין נחשבת רשימה וגודלה הוא 0.
- "מחרוזת ריקה" – מחרוזת שאינה מכילה תווים, כלומר **""** (מירכאות פותחות וסוגרות) או **''** (גרש פתיחה וסגירה). זוהי עדיין מחרוזת, ואורכה 0. מחרוזת המכילה לפחות רווח אחד אינה ריקה.

משימה:

- כתבו פונקציה אשר מקבלת מספרים רבים מהמשתמש, ומחזירה (return) רשימה בה כל הקלטים שהכניס המשתמש ובסופה סכומם. הפונקציה תקבל קלטים רבים מהמשתמש עד אשר יכניס מחרוזת ריקה.
- חתימת הפונקציה הינה:

```
def input_list():
```

הערות למימוש:

- הפונקציה תחזיר (return) רשימה המכילה את כל הקלטים שהמשתמש הכניס, כמספרים (ולא כמחרוזות), לפי הסדר. בסוף הרשימה, אחרי המספר האחרון שהזין המשתמש, יופיע סכום כל המספרים שהזין.
  - כל תא ברשימה יכיל בדיוק קלט אחד של המשתמש.
  - הקלט האחרון של המשתמש (המחרוזת הריקה) לא צריך להיות איבר ברשימה.
  - סידור הערכים ברשימה יהיה כך שהקלט הראשון יימצא בתא ה-0 ברשימה והקלט ה-n יימצא בתא ה-n-1 ברשימה. במקום ה-n ברשימה יימצא סכום כל המספרים.
- במקרה שהקלט הראשון הוא מחרוזת ריקה, תחזיר הפונקציה רשימה עם איבר יחיד 0.
- ניתן להניח שהמשתמש אכן יקליד רק מספרים.
- אין להשתמש בפונקציה **sum()**.
- מומלץ להשתמש בפונקציה **append**.
- הפונקציה **input** מחזירה מחרוזת, גם אם מקלידים מספר. אם המשתמש יקליד את המספר 371, אז הפונקציה תחזיר את המחרוזת "371" ולא את המספר 371. כדי לבצע המרה ממחרוזת למספר, השתמשו בפונקציה float().
- יש לקרוא לפונקציה **input** ללא מחרוזת (כלומר אין להדפיס למסך הודעה לבקשת קלט).

<u>דוגמה לשימוש בפונקציה</u> (הטקסט באדום הוא הסבר לתרגיל ולא מופיע בקלט המשתמש):

עבור הקלט:

311

0

-3

2

Enter with no input

הפונקציה תחזיר את הרשימה:

[311.0, 0.0, -3.0, 2.0, 310.0]

שימו לב, זוהי רשימה של מספרים!

חלק ב' – לולאות ורשימות

2. <u>חישוב מכפלה פנימית</u>

<u>רקע</u>:

בהמשך, בקורס אלגברה לינארית, תלמדו על מרחבי מכפלה פנימית. באופן כללי, זהו מרחב וקטורי שעליו מוגדרת פעולה הנקראת מכפלה פנימית (או בקיצור מ"פ) – פעולה שעונה על הגדרה מסוימת. אנו נממש מ"פ חשובה ומעניינת, הנקראת המ"פ הסטנדרטית על $\mathbb{R}^n$. אנו מתייחסים אל וקטורים כאל רשימות.

<u>הגדרה</u>:

בהינתן שני וקטורים (עבורנו, שתי רשימות) של מספרים: $[x_1, x_2, \dots, x_{n-1}, x_n], [y_1, y_2, \dots, y_{n-1}, y_n]$ המכפלה הפנימית הסטנדרטית שלהן מוגדרת להיות המספר הבא: $x_1 \cdot y_1 + x_2 \cdot y_2 + \cdots + x_n \cdot y_n$.

<u>משימה</u>:

- כתבו פונקציה המקבלת שתי רשימות של מספרים, ומחזירה את המ"פ הסטנדרטית שלהן.
- <u>חתימת הפונקציה</u>:

```
def inner_product(vec_1, vec_2):
```

- <u>פלט הפונקציה</u>: מספר.

<u>הערות למימוש</u>:

- ניתן להניח שהקלט הוא אכן שתי רשימות.
- ניתן להניח שהרשימות מכילות מספרים מסוג int או float בלבד.

- אם הרשימות אינן באותו האורך, הפונקציה צריכה להחזיר את הערך **None**.
- אם הרשימות ריקות, הפונקציה צריכה להחזיר את הערך 0.

3. <u>בדיקת מונוטוניות של סדרה סופית</u>

<u>הגדרה</u>:

בהינתן סדרת מספרים $(a_n)_{n=0}^{N}$ בעלת $N+1$ איברים $(a_0, a_1, a_2, \dots, a_{N-1}, a_N)$:

0. נאמר שהסדרה $(a_n)_{n=0}^{N}$ **מונוטונית עולה** אם לכל $1 \le n \le N$ מתקיים: $a_{n-1} \le a_n$.
1. נאמר שהסדרה $(a_n)_{n=0}^{N}$ **מונוטונית עולה ממש** אם לכל $1 \le n \le N$ מתקיים: $a_{n-1} < a_n$.
2. נאמר שהסדרה $(a_n)_{n=0}^{N}$ **מונוטונית יורדת** אם לכל $1 \le n \le N$ מתקיים: $a_{n-1} \ge a_n$.
3. נאמר שהסדרה $(a_n)_{n=0}^{N}$ **מונוטונית יורדת ממש** אם לכל $1 \le n \le N$ מתקיים: $a_{n-1} > a_n$.

<u>משימה</u>:

- כתבו פונקציה המקבלת רשימה (של מספרים בלשהם מסוג *float* או *int*, כלומר סדרה $(a_n)_{n=0}^{N}$ בלשהי) ומחזירה כפלט האם הסדרה עונה לכל אחת מההגדרות הנ"ל.
- <u>חתימת הפונקציה:</u>

def sequence_monotonicity(sequence):

- <u>פלט הפונקציה:</u> רשימה בעלת 4 איברים בוליאניים: המקום ה-0 ברשימה מתייחס להגדרה 0, מקום 1 להגדרה 1, וכך הלאה. אם הסדרה עונה להגדרה, במיקום המתאים ברשימה יוחזר הערך *True*. אחרת, *False*.

<u>דוגמאות לקלט ופלט:</u>

sequence_monotonicity ([1,2,3,4,5,6,7,8]) → [True, True, False, False]

sequence_monotonicity ([1,2,2,3]) → [True, False, False, False]

sequence_monotonicity ([7.5, 4, 3.141, 0.111]) → [False, False, True, True]

sequence_monotonicity ([1, 0, -1, 1]) → [False, False, False, False]

<u>הערות למימוש:</u>

- ניתן להניח שהקלט לפונקציה הוא אכן רשימה, וברשימה אין איברים שאינם מסוג *int* או *float*.
- בהינתן רשימה ריקה או רשימה בעלת איבר יחיד, הפונקציה תחזיר *[True, True, True, True]*.
- <u>אין להשתמש בפונקציות</u> *all, any*.

4. דוגמאות לסדרות

הקדמה:

דרך טובה להבנת הגדרה היא הסתכלות על דוגמאות. לכן כדי להבין בצורה אינטואיטיבית יותר אילו סדרות מקיימות אילו מארבעת ההגדרות הנ"ל, נממש פונקציה המספקת דוגמאות.

משימה:

- כתבו פונקציה המקבלת רשימה בת 4 איברים בוליאניים (קרי *True* או *False*), ומחזירה רשימה בת 4 מספרים המייצגת סדרה סופית שהיא בעצם דוגמה לסדרה המקיימת את ההגדרות בהתאם להיכן שיש True בקלט.
- חתימת הפונקציה:

def monotonicity_inverse(def_bool):

- פלט הפונקציה: רשימה בת 4 מספרים המייצגת סדרה סופית, כאשר הסדרה עונה להגדרה ה-i אם ורק אם ישנו *True* במקום ה-i בקלט (כלומר בהתאם ל-4 ההגדרות בסעיף הקודם. דוגמה בהמשך). אם אין רשימה כזאת, הפונקציה תחזיר **None**.

דוגמאות לקלטים ופלטים:

- monotonicity_inverse([True, True, False, False]) → [56.5, 57.5, 63, 84]
  - **זו אכן סדרה עולה ועולה ממש, אבל לא יורדת ולא יורדת ממש**
- monotonicity_inverse([False, True, False, False]) → **None**
  - **אין דוגמה לסדרה העונה להגדרות הנ"ל.**

הערות למימוש:

- ניתן להניח כי הקלט הוא רשימה בעלת 4 איברים בוליאניים (קרי True או False).
- ניתן לכתוב את התשובות המוחזרות במפורש (אין הכרח לחשבן בתוך הפונקציה).
- שימו לב: ישנם 2^4 קלטים חוקיים לפונקציה. חישבו עבור כמה מהם אין סדרות מתאימות.
- חישבו: עד כמה ניתן לוודא את נכונות הפונקציה בעזרת הסעיף הקודם?
- בסעיף זה אין הכרח להשתמש בלולאות.

חלק ג' – לולאות מקוננות

5. קונבולוציה על מטריצה דו מימדית

הקדמה:

קונבולוציה היא פעולה מתמטית המוגדרת באופנים והקשרים רבים. בשאלה זו נממש אופרטור קונבולוציה המשמש לראייה ממוחשבת ונפוץ בעיבוד תמונה קלאסי, בינה מלאכותית, ועוד.

הגדרות:

- **מטריצה:** רשימה של רשימות (ויזואלית, ניתן לחשוב עליהן כמלבן של מספרים).
- **האיבר ה i,j במטריצה:** המספר שממוקם בשורה ה i ובעמודה ה j (באשר 0,0 הוא האיבר השמאלי העליון).
- **אופרטור קונבולוציה:** פונקציה באשר מפעילים אותה על מטריצה נתונה, התוצאה היא בעצמה מטריצה באשר כל איבר בה הוא סכום של 9 איברים בתוך ריבוע של 3x3, ש"רץ" על מטריצת הקלט, משמאל לימין ממעלה עד מטה. כלומר, אם ניקח ריבוע בגודל 3 על 3 ,נמקם אותו בחפיפה מקצה השמאלי עליון של המטריצה המקורית ונסכום את האיברים של המטריצה המקורית שנכנסו בתוכו: נקבל את האיבר ה 0,0 במטריצת התוצאה. אם נסיט את הריבוע מקום אחד ימינה (ונסכום את האיברים החדשים הנכנסים בו) נקבל את האיבר 0,1 במטריצת התוצאה, וכך הלאה. ברגע שלא ניתן להזיז את הריבוע עוד ימינה על מטריצת הקלט המקורית, נחזיר אותו שוב לעמודה ה 0, ונוריד אותו שורה אחת למטה, ונחזור על התהליך.

<u>דוגמה לחישוב תוצאת קונבולוציה:</u>

1	1	1	1	0
1	1	0	0	0
0	0	1	0	1
0.5	0	0	1	0

→

6		

1	1	1	1	0
1	1	0	0	0
0	0	1	0	1
0.5	0	0	1	0

→

6	5	

1	1	1	1	0
1	1	0	0	0
0	0	1	0	1
0.5	0	0	1	0

→

6	5	4

1	1	1	1	0
1	1	0	0	0
0	0	1	0	1
0.5	0	0	1	0

→

6	5	4
3.5		

1	1	1	1	0
1	1	0	0	0
0	0	1	0	1
0.5	0	0	1	0

→

6	5	4
3.5	3	

1	1	1	1	0
1	1	0	0	0
0	0	1	0	1
0.5	0	0	1	0

→

6	5	4
3.5	3	3

משימה:

- כתבו פונקציה המקבלת מטריצה (רשימה של רשימות) ומחזירה את מטריצת התוצאה של אופרטור הקונבולוציה עליה (רשימה של רשימות).
- חתימת הפונקציה:

def convolve(mat):

דוגמאות לקלטים ופלטים:

convolve([[1, 1, 1, 1, 0], [1, 1, 0, 0, 0], [0, 0, 1, 0, 1], [0.5, 0, 0, 1, 0]]) → [[6, 5, 4], [3.5, 3, 3]]

convolve ([[1, 1, 1, 1], [1, 0, 1, 0], [0, 0, 1, 0], [0.5, 0, 0, 1], [2, 0, 0, 0]]) → [[6.0, 5.0], [3.5, 3.0], [3.5, 2.0]]

convolve ([[1, 1, 1, 1], [1, 1, 1, 1], [1, 1, 1, 1], [1, 1, 1, 1]]) → [[9, 9], [9, 9]]

convolve ([[1, 1, 1], [1, 1, 1], [1, 1, 1], [1, 1, 0]]) → [[9], [8]]

הערות למימוש:

- ניתן להניח כי הקלט הוא אכן רשימה של רשימות.
- אם הקלט הוא רשימה ריקה, הפונקציה תחזיר **None.**
- אחרת, ניתן להניח כי כל אחת מהרשימות שבקלט היא רשימת מספרים מסוג int או float, שהרשימות באותו האורך, ישנם לפחות 3 רשימות, ובכל רשימה יש לפחות 3 איברים (בלומר יתקבל לכל הפחות ריבוע בגודל 3 על 3).
- הערה למחשבה: האם ניתן לחזות מראש את גודל מטריצת הפלט? האם זה נחוץ בהכרח למימוש השאלה?

6. חיבור וקטורים

רקע:

כשתלמדו על מרחבים וקטורים, תיווכחו לדעת כי פעולת החיבור במרחב וקטורי היא פעולה חשובה ושימושית. זו בדיוק הפעולה שנממש, במרחב הוקטורי החשוב והידוע, $\mathbb{R}^n$, עם המ"פ הסטנדרטית.

הגדרה:

בהינתן שני וקטורים (עבורנו, שוב, שתי רשימות) של מספרים: $[x_1, x_2, \dots, x_{n-1}, x_n]$, $[y_1, y_2, \dots, y_{n-1}, y_n]$ נגדיר את החיבור הוקטורי של הרשימות להיות הרשימה: $[x_1 + y_1, x_2 + y_2, \dots, x_n + y_n]$ (בלומר חיבור קואורדינטה קואורדינטה).

משימה:

- כתבו פונקציה המקבלת רשימה של וקטורים (בלומר רשימה של רשימות) ומחזירה את הסכום הוקטורי שלהם.
- חתימת הפונקציה:

def sum_of_vectors(vec_lst):

- <u>פלט הפונקציה</u>: וקטור (רשימה) המהווה את סכום הוקטורים בקלט.

<u>דוגמאות לקלטים ופלטים:</u>

sum_of_vectors([[1,1], [1,3]])→ [2, 4]

sum_of_vectors([[1,1, 1], [1,0, 0], [0, 0, 100]])→ [2, 1, 101]

sum_of_vectors([[1,1,1,1,1], [1,1,1,1,1]])→ [2, 2, 2, 2, 2]

<u>הערות למימוש:</u>

- ניתן להניח כי הקלט הוא אכן רשימה של רשימות.
- אם הקלט הוא רשימה ריקה, הפונקציה תחזיר **None.**
- אם הרשימות בקלט ריקות, הפונקציה תחזיר רשימה ריקה**.**
- אחרת, ניתן להניח כי כל אחת מהרשימות שבקלט היא רשימת מספרים מסוג int או float, וכי הרשימות באותו האורך.

7. <u>בדיקת אורתוגונליות</u>

<u>רקע</u>:

בנאמר לעיל, לאחר שתלמדו על מרחבי מכפלה פנימית, תלמדו על המושג הנקרא אורתוגונליות. ההגדרה אומרת ששני וקטורים אורתוגונליים זה לזה, אם המכפלה הפנימית ביניהם היא 0. באופן כללי, אפשר לחשוב על זה כ-"וקטורים הניצבים זה לזה".

<u>הגדרה</u>:

בהינתן שתי רשימות של מספרים: $[x_1, x_2, \dots, x_{n-1}, x_n], [y_1, y_2, \dots, y_{n-1}, y_n]$ נאמר שהרשימות ניצבות זו לזו, אם מתקיים: $x_1 \cdot y_1 + x_2 \cdot y_2 + \cdots + x_n \cdot y_n = 0$.

<u>משימה:</u>

- כתבו פונקציה המקבלת רשימה של וקטורים (כלומר רשימה של רשימות) ומחזירה את **מספר הזוגות** של הרשימות הניצבות זו לזו.
- <u>חתימת הפונקציה:</u>

def num_of_orthogonal(vectors):

- <u>פלט הפונקציה</u>: מספר הזוגות של הרשימות הניצבות זו לזו ברשימה.

<u>דוגמה לקלטים ופלטים</u>:

num_of_orthogonal([[1,0,0], [0,1,0], [0,0,1]]) → 3

num_of_orthogonal([[0,0,0], [0,1,0], [0,0,1]]) → 3

num_of_orthogonal([[0, 0], [1, 2], [10, 5]]) → 2

num_of_orthogonal([[1,1,1,1], [2,1,3,3], [0, 0, 100, 33], [8, 8, 8, 1.5], [9,9,9,9]]) → 0

הערות למימוש:

- ניתן להניח כי הקלט הוא אכן רשימה.
- ניתן להניח שהרשימות באותו האורך, ושאינן מכילות ערכים שאינם מספרים (float או int).
- **שימו לב לכפילויות:** כל זוג סופרים **פעם אחת** ולא יותר. חישבו: האם ניתן להימנע מספירה כפולה? כיצד?
- השתמשו בפונקציה הקודמת שמימשתם לחישוב המכפלה הפנימית בין שתי רשימות. המנעו מכפילויות קוד ושמרו על קוד נאה.
- כשאתם בודקים את עצמכם, היזהרו לא להתבלבל משימוש במספרים לא שלמים. לאחר מספר מסויים של ספרות אחרי הנקודה הדיוק של פייתון אינו מובטח. התעלמו ממקרים כאלו, וכדי להבין למה זה קורה – חכו לקורסים הבאים.

הוראות הגשה

עליכם להגיש את הקובץ **ex3.zip** (בלבד) בקישור ההגשה של תרגיל 3 דרך אתר הקורס (moodle).

ההגשה היא עד המועד הנקוב בראש הקובץ.

ex3.zip צריך לכלול את הקבצים הבאים בלבד:

1. ex3.py

ודאו שבקובץ ה-py. שאתם מגישים אין קריאות הרצה לפונקציות שכתבתם.

הנחיות כלליות בנוגע להגשה

- הנכם רשאים להגיש תרגילים דרך מערכת ההגשות באתר הקורס מספר רב של פעמים. ההגשה האחרונה בלבד היא זו שקובעת ושתיבדק.
- לאחר הגשת התרגיל, ניתן ומומלץ להוריד את התרגיל המוגש ולוודא כי הקבצים המוגשים הם אלו שהתכוונתם להגיש וכי הקוד עובד על פי ציפיותיכם.
  - באחריותכם לוודא כי – PDF הבדיקות נראה כמו שצריך.
- קראו היטב את קובץ נהלי הקורס לגבי הנחיות נוספות להגשת התרגילים.

בהצלחה!